

Quranic Chatbot

Project Handbook

Sammar Abbas Shah

Flutter app + FastAPI backend | Version 1.0.0

Table of Contents

1. What the app is.....	5
2. How it all fits together.....	5
3. Setup at a glance.....	6
4. Tools to install on your PC.....	6
4.1 Git.....	6
4.2 Flutter SDK.....	6
4.3 Android Studio (provides the Android SDK and Java).....	7
4.4 Visual Studio Code (editor).....	7
4.5 Python (for the backend).....	8
4.6 Node.js and Vercel CLI (optional).....	8
4.7 An Android phone.....	8
4.8 Final check.....	8
5. Accounts and API keys you need.....	9
6. The Flutter app.....	10
6.1 What is in the project.....	10
6.2 How the source code is organised.....	10
6.3 Packages used.....	12
6.4 First-time project setup.....	12
6.5 Android permissions.....	12
6.6 Optional: bundled offline Quran.....	13
6.7 Screens and routes.....	13
6.8 App flow.....	13
6.9 Real vs mock services.....	13
7. The backend (FastAPI).....	14
7.1 Chat: how the answer is produced.....	14
7.2 Speech-to-text (/stt).....	14
7.3 Recite & Recognize (/recognize).....	14
7.4 Caching.....	15
7.5 Python dependencies (requirements.txt).....	15

7.6 Running the backend on your own PC (optional).....	15
8. Deploying to Vercel and setting environment variables.....	15
8.1 Where the original backend lives.....	15
8.2 The two configuration files.....	16
8.3 Deploy it.....	16
8.4 Environment variables.....	17
8.5 Updating the backend.....	18
8.6 Testing the deployment.....	18
9. Connecting the app to the backend.....	19
10. Feature guide.....	19
10.1 Reading the Quran.....	19
10.2 Recitation and Auto Play.....	19
10.3 Offline use.....	20
10.4 AI chat.....	20
10.5 Voice chat.....	20
10.6 Recite & Recognize.....	20
10.7 Bookmarks and history.....	21
10.8 Settings.....	21
10.9 Accounts.....	21
11. Data stored on the phone.....	21
12. Running, building and sharing the APK.....	22
12.1 Run on a phone (debug).....	22
12.2 Build the release APK.....	22
12.3 Sending the APK to someone.....	22
13. Troubleshooting.....	22
14. Security and religious-content notes.....	24
15. Known limitations and future work.....	24
16. Quick command reference.....	24

Text in angle brackets, such as `<YOUR - PROJECT>`, is a value you replace with your own. **Never write real API keys in code, documents or screenshots.**

About the pictures in this handbook. The screen pictures are illustrations drawn to show *where to click*. Windows, Android Studio, GitHub and Vercel update their looks from time to time, so your screen may differ slightly; the menu names and the order of the steps are what matter. Red numbered circles show the order in which to click.

1. What the app is

Quranic Chatbot is an Android app (built with Flutter, Material 3) for reading, listening to, searching and asking questions about the Holy Quran. It has a Flutter front end and a small FastAPI backend that talks to the AI and Quran data services.

Area	What it does
Quran reader	Browse 114 Surahs and 30 Juz. Arabic text with English and Urdu translation.
Recitation audio	Tap a verse to hear it (Mishary Alafasy, 128 kbps). Verses are saved on first play and then work offline.
Auto Play	When on, the next verse plays automatically and its card is highlighted. Off by default.
Search	Search verses by keyword.
AI chat	Ask a question, get a short answer with verse citations.
Voice chat	Speak a question; it is converted to text, then answered.
Recite & Recognize	Recite a verse; the app identifies it and shows Arabic + English + Urdu.
Bookmarks and History	Saved on the device.
Settings	12 themes plus system theme, language (English / Urdu / Arabic), font size, translation options, reciter, playback speed, Auto Play.

The interface supports English, Urdu and Arabic, including right-to-left layout.

2. How it all fits together

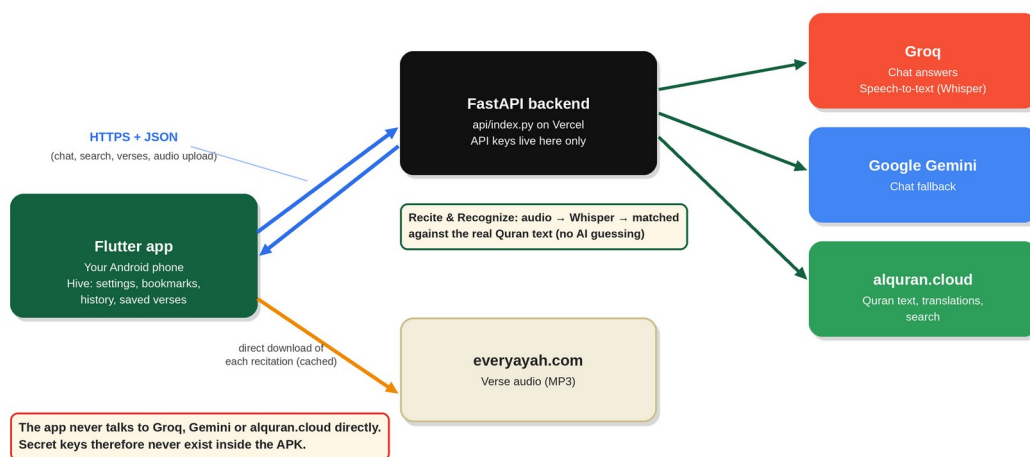


Figure 1: How the phone, the backend and the outside services connect

Why a backend at all? An app must never contain secret API keys, because anyone can extract them from an APK. The keys live only on Vercel as environment variables. The app talks to the backend, and the backend talks to Groq and Gemini.

Verse recognition is not an AI guess. Whisper (via Groq) turns the recitation audio into Arabic text. The backend then compares that text with the real Quran text using string similarity and returns the closest verse plus a real confidence score. The AI never invents a surah or ayah number.

3. Setup at a glance

Follow these steps in order. Each one is explained in detail in the sections that follow.

1. Install the tools on your PC: Git, Flutter, Android Studio, VS Code, Python (section 4).
2. Create the accounts and API keys: GitHub, Vercel, Groq, Gemini (section 5).
3. Deploy the backend on Vercel and add the two API keys as environment variables (section 8). You need its URL before the app can work.
4. Turn the `flutter_app` folder into a runnable Android project and add the microphone/internet permissions (section 6.5).
5. Put your Vercel URL into `lib/services.dart` (section 9).
6. Optional: build the offline Quran file (section 6.6).
7. Run the app on a phone, then build a release APK (section 12).

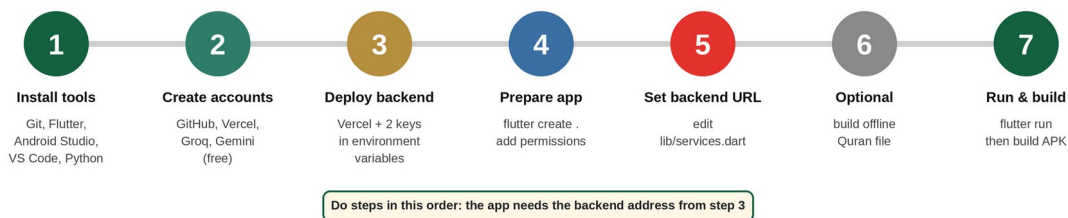


Figure 2: The seven setup steps in order

4. Tools to install on your PC

The steps below use **Windows**, which is what this project was built on. On macOS or Linux the tools are the same; only paths and installers differ. Install in this order.

4.1 Git

- Download from git-scm.com and install with the default options.
- Needed to push the backend to GitHub, and Flutter itself relies on it.
- Check: `git --version`

4.2 Flutter SDK

- Download the stable Windows SDK from docs.flutter.dev/get-started/install.
- Extract it to a short path with no spaces, for example `C:\src\flutter` (do not put it in a folder like `Program Files`).
- Add the SDK's `bin` folder (for example `C:\src\flutter\bin`) to your Windows **PATH**: Start → “Edit the system environment variables” → Environment Variables → Path → New.
- Check: `flutter --version`
- Flutter includes the Dart SDK, so Dart does not need a separate install.

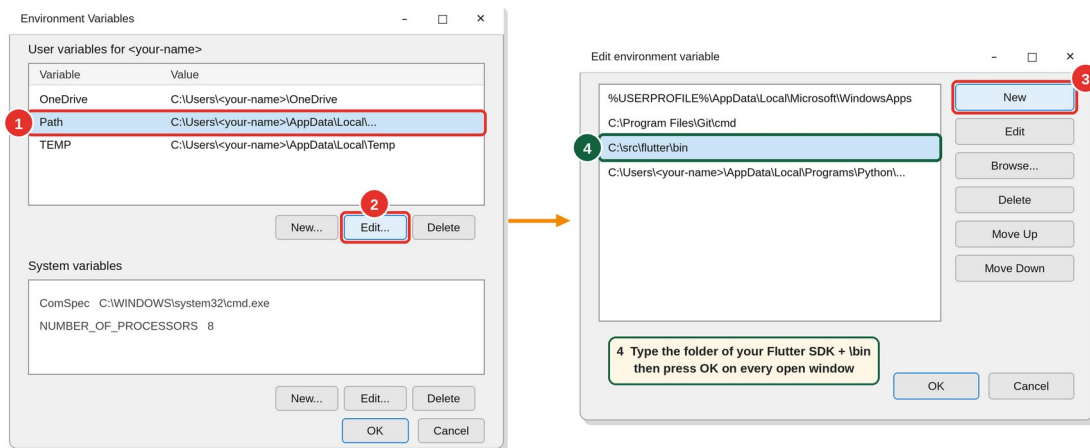
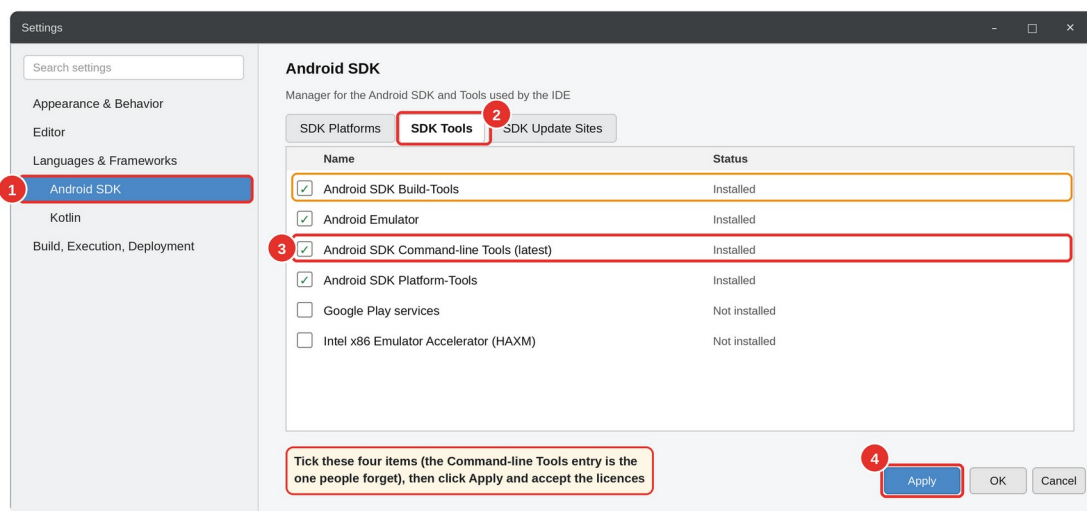


Figure 3: Adding the Flutter folder to the Windows Path (Start → “Edit the system environment variables” → Environment Variables)

4.3 Android Studio (provides the Android SDK and Java)

- Install Android Studio from developer.android.com/studio.
- On first launch, let the setup wizard install the **Android SDK**, **SDK Platform-Tools** and **Emulator**.
- Open **Settings** → **Languages & Frameworks** → **Android SDK** → **SDK Tools** and tick:
 - Android SDK Build-Tools
 - **Android SDK Command-line Tools (latest)**
 - Android SDK Platform-Tools
- Accept the licences: `flutter doctor --android-licenses` (press **y** repeatedly).
- Android Studio also ships the JDK that Gradle uses, so no separate Java install is needed.



- The

project builds with **Gradle 9.3.1**, which is downloaded automatically on the first build. The first build takes several minutes.

4.4 Visual Studio Code (editor)

- Install VS Code from code.visualstudio.com.
- Install the extensions **Flutter** and **Dart** (Dart is added automatically with Flutter).
- Optional but useful: **Python** (for `api/index.py`) and **GitLens**.

- This is *VS Code*, not “Visual Studio”. Full Visual Studio is not needed for an Android-only app (it is only required if you later build a Windows desktop version).

4.5 Python (for the backend)

- Install Python 3.10 or newer from python.org. Tick “**Add Python to PATH**” in the installer.
- Needed for the helper scripts in `flutter_app/tools` and to run the backend on your own PC. Vercel runs the backend in the cloud, so Python is not needed for deployment.
- Check: `python --version`

4.6 Node.js and Vercel CLI (optional)

- Only needed to deploy from the terminal instead of through the GitHub integration.
- Install Node.js LTS from nodejs.org, then run: `npm install -g vercel`

4.7 An Android phone

- Enable **Developer options**: Settings → About phone → tap *Build number* 7 times.
- Enable **USB debugging**.
- Connect by USB and accept the “Allow USB debugging” prompt.
- Check that Flutter sees the phone: `flutter devices`

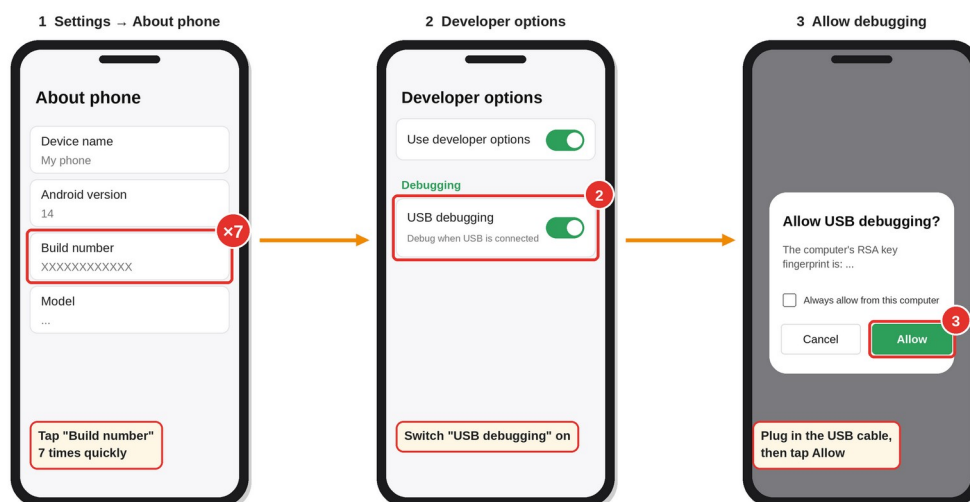


Figure 5: Turning on USB debugging on an Android phone

4.8 Final check

Run this and fix everything that shows a red **X**:

```
flutter doctor -v
```

You should see green ticks for Flutter, Android toolchain, Connected device and VS Code. Warnings about Chrome, Windows or Visual Studio can be ignored for an Android-only project.

```

PS C:\dev> flutter doctor -v
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.x.x, on Microsoft Windows ...)
[✓] Windows Version (Installed version of Windows is ...)
[✓] Android toolchain - develop for Android devices (Android SDK version 3x.x.x)
[✓] Chrome - develop for the web
[!] Visual Studio - develop Windows apps <- fine for an Android-only app
[✓] Android Studio (version 20xx.x)
[✓] VS Code (version 1.xx)
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 1 category.

```

Any red [X] line must be fixed first. Warnings about Chrome, Windows or Visual Studio can be ignored for an Android-only app.

Figure 6: What a healthy flutter doctor result looks like (version numbers will differ)

5. Accounts and API keys you need

Service	Used for	Cost	Where to get it
GitHub	Stores the backend code; Vercel deploys from it	Free	github.com
Vercel	Hosts the backend	Free (Hobby) plan	vercel.com (sign in with GitHub)
Groq	Chat answers (primary) and speech-to-text (Whisper)	Free tier	console.groq.com → API Keys → Create
Google AI Studio (Gemini)	Chat answers (fallback)	Free tier	aistudio.google.com → Get API key

Free public APIs used (no key needed):

API	Used for
api.alquran.cloud	Surah list, Arabic text (quran-uthmani), English (en.sahih), Urdu (ur.jalandhry), search
everyayah.com	Verse audio (Alafasy_128kbps)

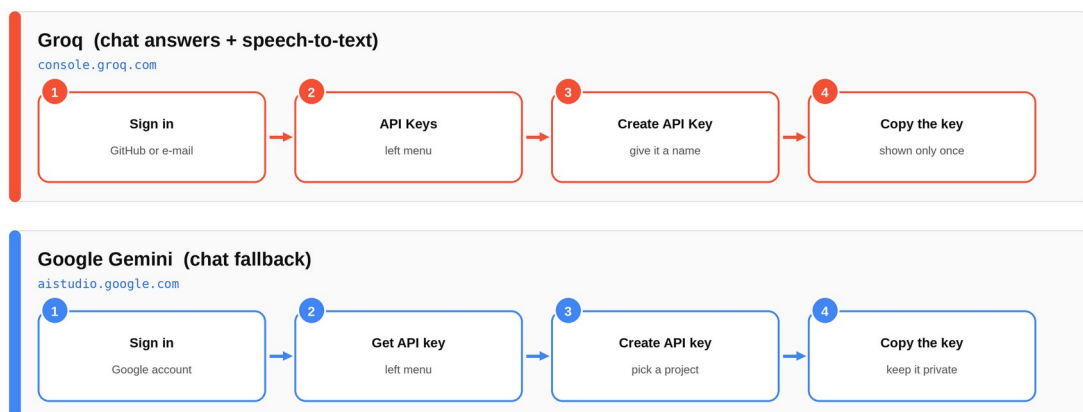


Figure 7: Where to create the two API keys (copy each key straight into Vercel; never save it elsewhere)

Key rules

- Create the two keys and paste them **only** into Vercel's environment variables (section 8).

- Never put a key in `api/index.py`, in the Flutter code, in GitHub, in screenshots or in documents.
- If a key ever leaks, delete it in the provider's console and create a new one.

6. The Flutter app

6.1 What is in the project

```

quranic_chatbot_project/
|-- README.md           quick start
|-- Quranic_Chatbot_Handbook.pdf  this handbook (also .docx)
|-- flutter_app/        the mobile app
|   |-- pubspec.yaml    packages and assets
|   |-- lib/
|       |-- main.dart   entry point: imports, file list, main()
|       |-- services.dart HTTP layer + backend URL (edit this)
|       |-- app/        app root, dependency container, routes
|       |-- models/     data classes
|       |-- data/       built-in Surah names, Juz positions, sample verses
|       |-- utils/      helper functions
|       |-- services/   service contracts, mock + real services, audio, cache
|       |-- state/     controllers (settings, chat, bookmarks, ...)
|       |-- l10n/      UI text in English / Urdu / Arabic
|       |-- theme/     colours, typography, the 12 themes
|       |-- widgets/   reusable widgets
|       |-- screens/   full-page screens
|   |-- assets/        (offline Quran goes here, optional)
|   |-- tools/         helper scripts (offline Quran, Android permissions)
|-- backend/          the FastAPI server that is deployed to Vercel
|   |-- api/index.py  the whole backend
|   |-- vercel.json   tells Vercel how to run it
|   |-- requirements.txt Python packages
|   |-- .gitignore

```

The Android folder is not included because Flutter generates it for you (section 6.4).

6.2 How the source code is organised

`lib/main.dart` lists every other file in `lib/` with a `part` line, and each of those files begins with `part of '../main.dart'`; This means all the files together form **one Dart library**: they share the same imports (which are all declared once in `main.dart`) and can use each other's classes directly, with no extra `import` lines. When you add a new file, create it with `part of '../main.dart'`; at the top and add a matching `part 'folder/file.dart';` line in `main.dart`.

Folder	What is inside	Files
<code>app/</code>	App root and wiring	<code>app.dart</code> (MaterialApp, theme, <code>AppServices</code> = which service is used), <code>routes.dart</code> (named routes, page transition)
<code>models/</code>	Data classes	<code>quran_models.dart</code> (Surah, Ayah, Translation), <code>chat_models.dart</code> , <code>bookmark_model.dart</code> , <code>recognition_models.dart</code> , <code>audio_models.dart</code> , <code>user_model.dart</code> , <code>enums.dart</code>
<code>data/</code>	Built-in data	<code>surah_catalog.dart</code> (114 Surahs), <code>juz_catalog.dart</code> , <code>mock_quran_data.dart</code> (sample verses), <code>featured_verses.dart</code>

Folder	What is inside	Files
<code>utils/</code>	Helpers	<code>helpers.dart</code> (script detection, Arabic normalising, date/duration formats), <code>autoplay_helpers.dart</code>
<code>services/</code>	Talking to the outside world	<code>service_interfaces.dart</code> (contracts), <code>mock_services.dart</code> , <code>backend_services.dart</code> (Quran and chat via the backend), <code>http_services.dart</code> (speech-to-text and verse recognition), <code>just_audio_service.dart</code> (audio + Auto Play), <code>recitation_cache.dart</code> (offline MP3s), <code>local_storage_services.dart</code> (bookmarks and history in Hive)
<code>state/</code>	Controllers that hold screen state	<code>settings_controller.dart</code> , <code>chat_controller.dart</code> , <code>voice_chat_controller.dart</code> , <code>recognition_controller.dart</code> , <code>quran_controller.dart</code> , <code>bookmarks_controller.dart</code> , <code>history_controller.dart</code> , <code>navigation_state.dart</code>
<code>l10n/</code>	Text	<code>strings.dart</code> (every UI string in three languages)
<code>theme/</code>	Look and feel	<code>theme_catalog.dart</code> (the 12 themes), <code>app_colors.dart</code> , <code>app_typography.dart</code> , <code>app_theme.dart</code>
<code>widgets/</code>	Reusable widgets	<code>ui_primitives.dart</code> , <code>feedback_widgets.dart</code> , <code>action_widgets.dart</code> , <code>audio_widgets.dart</code> , <code>verse_widgets.dart</code> (verse card), <code>chat_widgets.dart</code>
<code>screens/</code>	One file per screen (or small group)	<code>splash_screen.dart</code> , <code>onboarding_screen.dart</code> , <code>auth_screens.dart</code> , <code>main_shell.dart</code> , <code>home_screen.dart</code> , <code>quran_list_screen.dart</code> , <code>surah_screen.dart</code> , <code>juz_screen.dart</code> , <code>ayah_screen.dart</code> , <code>search_screen.dart</code> , <code>chat_screen.dart</code> , <code>voice_chat_screen.dart</code> , <code>verse_recognition_screen.dart</code> , <code>bookmarks_screen.dart</code> , <code>history_screen.dart</code> , <code>settings_screen.dart</code> , <code>about_screen.dart</code> , <code>surah_list_screen.dart</code> (backend test)

Where do I change...?

I want to change	Open
The backend address	<code>lib/services.dart</code>
Any wording or add a language string	<code>lib/l10n/strings.dart</code>

I want to change	Open
Colours or add a theme	<code>lib/theme/theme_catalog.dart</code>
How a verse card looks	<code>lib/widgets/verse_widgets.dart</code>
Auto Play behaviour	<code>lib/services/just_audio_service.dart</code> and the Surah/Juz screens
Which real/mock service is used	<code>AppServices</code> in <code>lib/app/app.dart</code>
Add a screen	new file in <code>lib/screens/</code> , then a route in <code>lib/app/routes.dart</code>
Default settings	<code>lib/state/settings_controller.dart</code>

6.3 Packages used

Package	Purpose
<code>hive_flutter</code>	Local database (settings, bookmarks, history, offline Quran cache)
<code>http</code>	Calling the backend
<code>just_audio</code>	Playing verse recitation
<code>record</code>	Recording the user's voice (voice chat and Recite & Recognize)
<code>path_provider</code>	Folder for saved recitation MP3s

They are already listed in `pubspec.yaml`; `flutter pub get` installs them.

6.4 First-time project setup

Run these once, inside the `flutter_app` folder:

```
cd flutter_app
flutter create . --platforms=android --project-name quranic_chatbot
flutter pub get
```

`flutter create .` generates the missing `android/` folder and leaves your `lib/` and `pubspec.yaml` alone. If Flutter created a `test/widget_test.dart` that fails to analyse, delete that file; the app does not use it.

6.5 Android permissions

The app needs internet access and the microphone. Run the helper (safe to repeat):

```
python tools/add_android_permissions.py
```

or add these two lines yourself to `android/app/src/main/AndroidManifest.xml`, just above `<application>`:

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
```

The microphone permission is requested when the user first taps record. If it is denied, recording features show a permission message. If the build complains that the `record` package needs a higher minimum SDK, set `minSdk = 23` in `android/app/build.gradle` (or `build.gradle.kts`).

6.6 Optional: bundled offline Quran

By default each Surah is downloaded from the backend the first time it is opened and then kept on the phone. To make **every** Surah available offline from the very first launch, build a bundled copy once your backend is deployed:

```
python tools/build_offline_quran.py https://<YOUR-PROJECT>.vercel.app
```

This writes `assets/quran_offline.json` (about 10 MB). Then open `pubspec.yaml`, remove the `#` in front of the `assets:` lines, and run `flutter pub get`. On first launch the app copies this file into its local database (Hive box `quran_cache`). Do **not** uncomment the asset lines before the file exists, or the build fails with “unable to find asset”.

6.7 Screens and routes

Route	Screen
<code>/</code>	Splash
<code>/onboarding</code>	Welcome cards (first launch)
<code>/login, /signup</code>	Local accounts
<code>/home</code>	Home
<code>/quran</code>	Surah / Juz list
<code>/surah, /juz, /ayah</code>	Readers
<code>/search</code>	Verse search
<code>/chat</code>	AI chat
<code>/voice-chat</code>	Voice chat
<code>/verse-recognition</code>	Recite & Recognize
<code>/bookmarks, /history</code>	Saved items
<code>/settings, /about</code>	Settings, About
<code>/backend-test</code>	Developer screen for testing the backend connection

6.8 App flow

Splash → (first launch) Onboarding → Login / Signup → Home.

The app remembers `isFirstLaunch` and `isLoggedIn` in Hive, so returning users go straight to Home.

6.9 Real vs mock services

`AppServices` (in `main.dart`) decides which implementation is used:

Service	Implementation
Quran data	<code>BackendQuranService</code> (backend first, built-in sample data as fallback)
Chat	<code>BackendChatService</code> → <code>/chat</code>
Speech-to-text	<code>HttpSpeechRecognitionService</code> → <code>/stt</code>
Verse recognition	<code>HttpVerseRecognitionService</code> → <code>/recognize</code>

Service	Implementation
Audio player	<code>JustAudioService</code> (real playback)
Bookmarks and history	<code>HiveBookmarkService</code> , <code>HiveHistoryService</code> (saved on the device)
Text-to-speech	<code>MockTextToSpeechService</code> (simulated; it does not actually speak)

7. The backend (FastAPI)

File: `backend/api/index.py`. It exposes these endpoints:

Method	Path	What it does
GET	<code>/surahs</code>	List of all 114 Surahs (cached in memory)
GET	<code>/surahs/{surah_id}/ayahs</code>	Arabic + English + Urdu + audio URL for every verse of a Surah
GET	<code>/juz/{juz_id}</code>	Same, for a Juz (1–30), fetched as three editions and combined
GET	<code>/search?q=...</code>	Keyword search, top 20 matches
POST	<code>/chat</code>	AI answer with citations
POST	<code>/stt</code>	Speech-to-text (upload an audio file)
POST	<code>/recognize</code>	Identify a recited verse (upload an audio file)

7.1 Chat: how the answer is produced

1. The app sends `{ "message": "...", "language": "en" }`.
2. The backend asks **Groq** (`openai/gpt-oss-120b`) for a compact JSON reply: `{"text": "...", "citations": ["2:255"]}`.
3. If Groq fails or returns something unusable, it tries **Gemini** (`gemini-3.6-flash`).
4. If both fail, it returns a friendly fallback sentence in English, Urdu or Arabic.
5. `_parse_json_reply()` repairs truncated or markdown-wrapped model output so a badly formatted reply doesn't crash the request.

Model names change over time. The two names are set at the top of `api/index.py` (`GROQ_CHAT_MODEL` and `GEMINI_CHAT_MODEL`). If chat suddenly stops working, check that both models are still offered by the provider and update the names if needed.

7.2 Speech-to-text (`/stt`)

Uploads the recorded audio to Groq's `whisper-large-v3` and returns `{ "text": "..." }`.

7.3 Recite & Recognize (`/recognize`)

1. Whisper transcribes the audio as Arabic.
2. `_normalize_arabic()` strips diacritics, unifies letter forms (ل → ل, etc.) and removes spaces so Whisper's spelling can be compared with the Uthmani script.
3. The whole Quran (6,236 verses) is loaded once from `alquran.cloud` and kept in memory.
4. Every verse is compared to the transcript with `difflib.SequenceMatcher`. The best match wins.

5. If the best score is below **0.45**, the API returns 404 **no_match**.
6. The response contains the transcript, the confidence, and the verse: **surah_id**, **ayah_number**, **arabic**, **english**, **urdu**, **audio_url**.

Error codes the app understands: **400 no_speech**, **404 no_match**, **500 speech_to_text_failed**, **503 index_unavailable**.

7.4 Caching

Surah lists, Surah verses, Juz data and the full Quran text are cached in server memory. On Vercel this cache lasts only while the serverless function stays warm, so the **first request after idle time is slower** (especially **/recognize**, which must download the whole Quran text).

7.5 Python dependencies (**requirements.txt**)

```
fastapi
httpx
groq
google-genai
python-multipart
```

python-multipart is required for file uploads (**/stt**, **/recognize**). FastAPI brings in **pydantic** automatically.

7.6 Running the backend on your own PC (optional)

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pip install uvicorn
set GROQ_API_KEY=<YOUR_GROQ_KEY>
set GEMINI_API_KEY=<YOUR_GEMINI_KEY>
uvicorn api.index:app --reload
```

Then open **http://127.0.0.1:8000/docs** for FastAPI's built-in test page. The server refuses to start if either key is missing (by design).

8. Deploying to Vercel and setting environment variables

8.1 Where the original backend lives

The backend that is currently hosted is published on GitHub:

<https://github.com/sammar-abbas-shah/quran-backend>

File in that repository	Role
api/index.py	The live backend. Vercel runs this file.
vercel.json	Tells Vercel to build api/index.py with the Python runtime and send every URL to it
requirements.txt	Python packages Vercel installs

File in that repository	Role
<code>.gitignore</code>	See the note in section 8.5
<code>main.py</code> (repository root)	An older, smaller copy of the backend (no <code>/juz</code> , <code>/stt</code> or <code>/recognize</code>). Vercel does not use it.

The `backend/` folder in this project is a copy of that layout, and its `api/index.py` matches the live file.

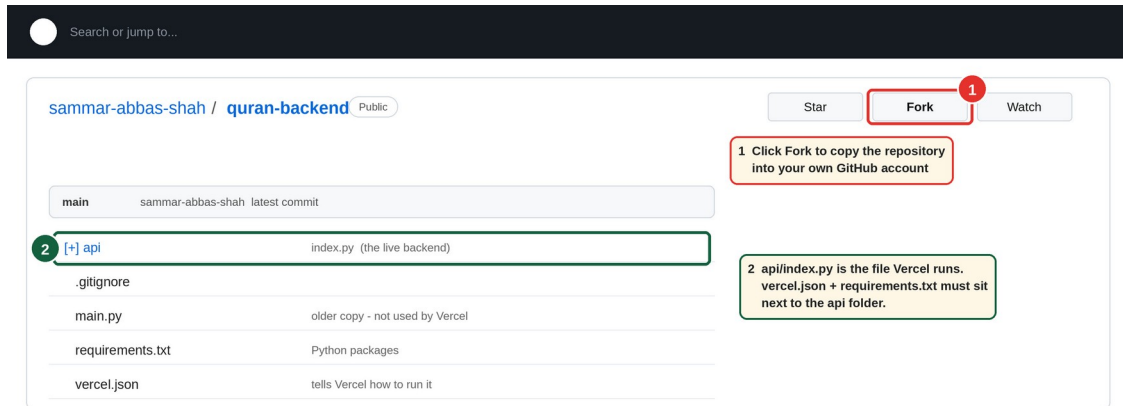


Figure 8: The original backend repository on GitHub: fork it, and note which file Vercel runs

8.2 The two configuration files

`vercel.json`:

```
{
  "builds": [{ "src": "api/index.py", "use": "@vercel/python" }],
  "routes": [{ "src": "/(.*)", "dest": "api/index.py" }]
}
```

`requirements.txt`:

```
fastapi
httpx
groq
google-genai
python-multipart
```

`python-multipart` is required for the audio upload endpoints. `uvicorn` is not listed because Vercel does not need it; install it only if you run the server on your own PC (section 7.6).

8.3 Deploy it

Choose one route.

Route A: use your own copy of the original repository

1. Sign in to GitHub and open the repository link above.
2. Click **Fork** to copy it to your account.
3. Go to `vercel.com` → **Add New** → **Project** → import your fork.

Route B: start from the project zip

1. Create a new empty repository on GitHub.

2. Copy the contents of the `backend/` folder into it (so `api/index.py` and `vercel.json` sit at the top level) and push.
3. Go to `vercel.com` → **Add New** → **Project** → import that repository.

Then, for either route:

1. Leave the framework preset on **Other**. The build settings come from `vercel.json`.
2. Add the environment variables from section 8.4 before you deploy.
3. Click **Deploy**. Your API URL will look like `https://<YOUR-PROJECT>.vercel.app`.

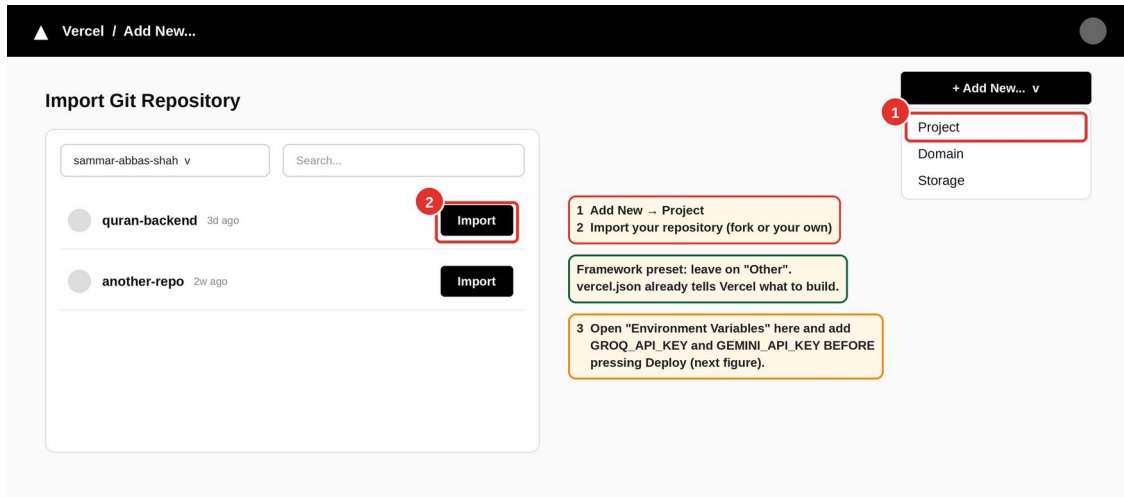


Figure 9: Vercel → Add New → Project → Import the repository

8.4 Environment variables

Project → **Settings** → **Environment Variables** → **Add**

Name	Value	Environments
<code>GROQ_API_KEY</code>	your Groq key	Production, Preview, Development
<code>GEMINI_API_KEY</code>	your Gemini key	Production, Preview, Development

`api/index.py` reads them with `os.environ.get(...)` and raises an error at start-up if either is missing.

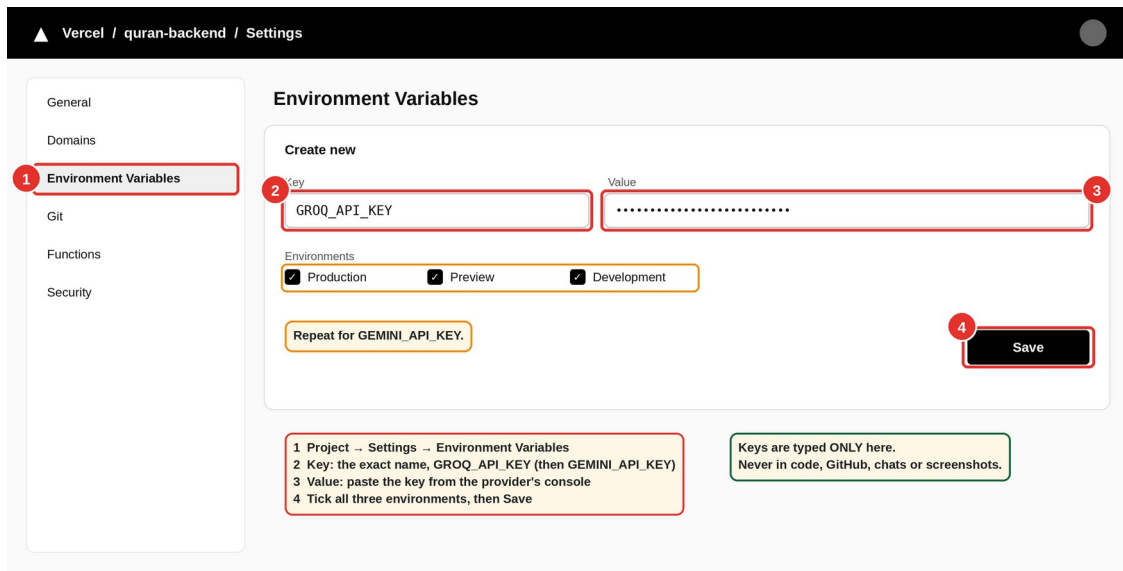


Figure 10: Adding an environment variable in Vercel (repeat for the second key)

Redeploy after changing variables. Environment variables are applied only to **new** deployments. After adding or changing a key: **Deployments** → **latest** → **...** → **Redeploy**.

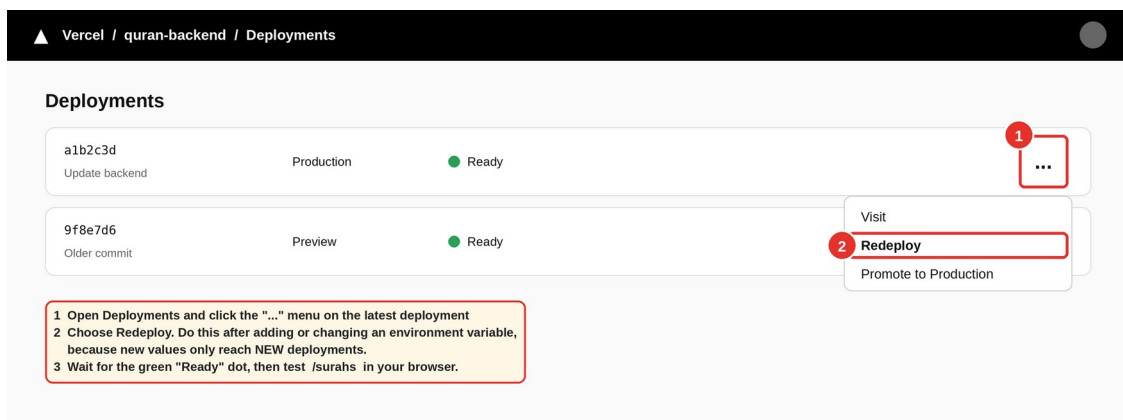


Figure 11: Redeploying so that new environment variables take effect

8.5 Updating the backend

Edit `api/index.py` → commit → push to GitHub → Vercel redeploys automatically. Check **Deployments** for a green “Ready” status, and **Logs** if something fails.

Note on `.gitignore`: the original repository’s `.gitignore` ignores everything (*) and then re-allows only `main.py`, `requirements.txt` and `.gitignore`. Files that Git already tracks (such as `api/index.py`) still update normally, but a brand-new file is skipped by `git add ..` Add such a file with `git add -f <file>`, or replace that `.gitignore` with the simpler one in `backend/.gitignore`.

8.6 Testing the deployment

Open these in a browser:

```
https://<YOUR-PROJECT>.vercel.app/surahs
https://<YOUR-PROJECT>.vercel.app/docs
```

`/surahs` should show a JSON list of 114 Surahs. `/docs` lets you try every endpoint, including `/chat`.

9. Connecting the app to the backend

The app's HTTP layer is `flutter_app/lib/services.dart`. At the top of that file is one constant:

```
const String kBackendBaseUrl = 'https://YOUR-PROJECT.vercel.app';
```

Replace it with your own Vercel URL (no trailing slash), then rebuild. If the Vercel URL ever changes (for example a new project is created), update it, rebuild the app and distribute the new APK.

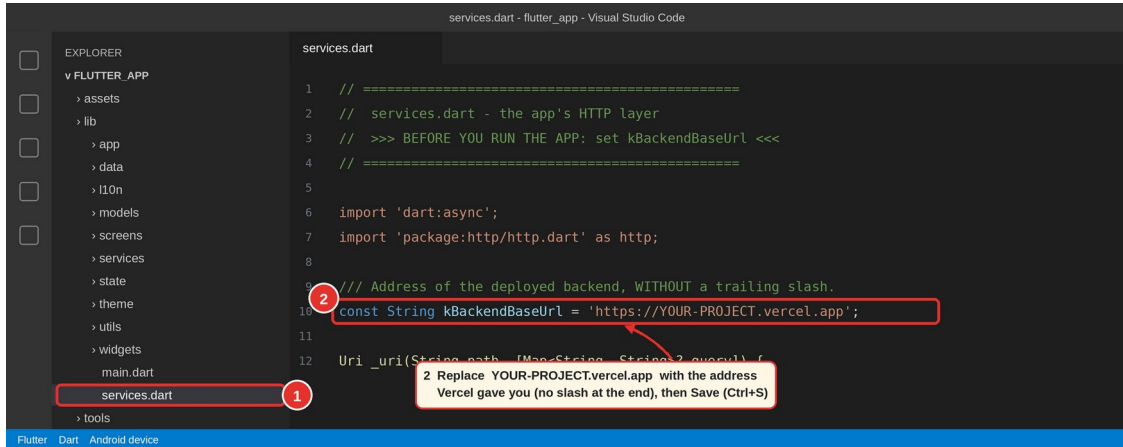


Figure 12: Setting the backend address in `lib/services.dart` (VS Code)

Use the in-app **Backend Test** screen (`/backend-test`) to confirm the app can reach the server before debugging anything else.

Notes: both an emulator and a real phone need internet access. A `localhost` address only works if the backend runs on your PC and the app is pointed at your PC's local-network IP.

10. Feature guide

10.1 Reading the Quran

- **Quran tab:** Surah list and Juz list.
- **Surah screen:** each verse is a card with Arabic, English and Urdu (toggle in Settings). The reading position is remembered (`lastSurah`, `lastAyah`, `lastJuz`).
- **Search results and Recognize results** can jump to a specific verse; that verse's card is highlighted.

10.2 Recitation and Auto Play

- Tap play on a verse. The recitation MP3 is downloaded and saved in the app's private folder `recitations/`, so it works offline next time.
- **Auto Play (Settings, off by default):** when a verse finishes, the next verse starts immediately, its card is highlighted and the list scrolls to it. It continues to the end of the Surah or Juz.
- The playing card is shown with a gold tint, a thicker border and a glow, and stays highlighted while paused mid-verse.
- While a verse plays, the next one is downloaded in the background so the hand-off has no gap. Verses without audio are skipped, and a load error shows a message instead of failing silently.

- Playback speed options: 0.75×, 1.0×, 1.25×, 1.5×.
- Settings shows how many recitation files are saved and their size, and lets you manage the cache.

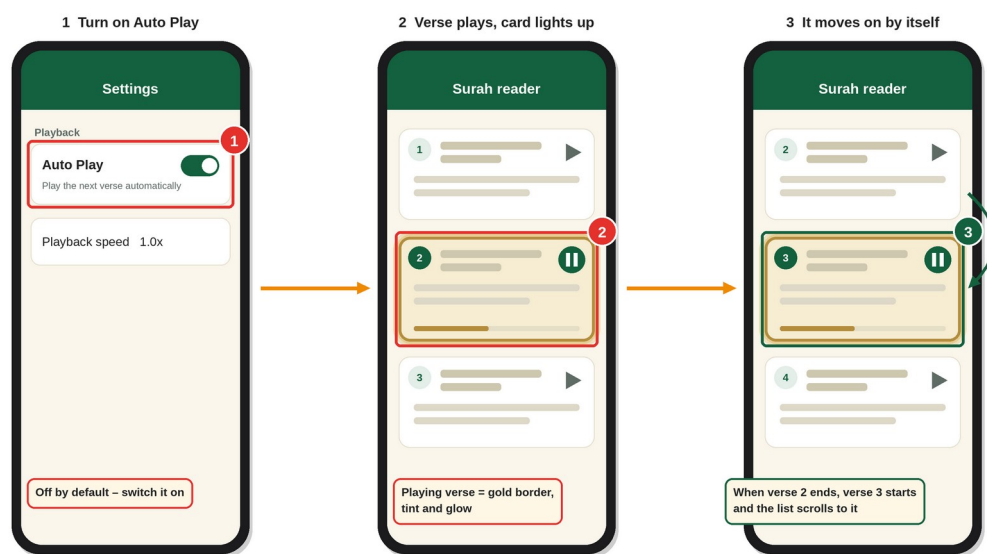


Figure 13: Auto Play: turn it on, the playing verse is highlighted, and the next verse starts by itself

10.3 Offline use

- If you built the optional `assets/quran_offline.json` (section 6.6), it is unpacked into the Hive box `quran_cache` on first launch, so all Surahs open without internet. Without it, each Surah is saved the first time it is opened online and then works offline.
- Recitation audio works offline only for verses that have been played at least once.
- Chat, voice chat and Recite & Recognize need internet (they call the backend).

10.4 AI chat

Type a question. The answer appears with **citation cards** (for example **2:255**) that open the verse. Answers are stored in History. The response language follows the *Response language* setting.

10.5 Voice chat

Record a question → `/stt` converts it to text → `/chat` answers. The spoken reply uses a simulated text-to-speech service, so replies are shown as text (see section 15).

10.6 Recite & Recognize

1. Open **Recite & Recognize** and allow the microphone.
2. Recite one verse clearly and stop recording.
3. The app shows *Your recitation* (the transcript), the *Matched verse* card with **Arabic, English and Urdu translation**, and a confidence percentage.
4. **View in Quran** opens that verse in its Surah. **Try again** resets.

Tips for better results: use a quiet room, recite a full verse (very short snippets match poorly) and hold the phone close.

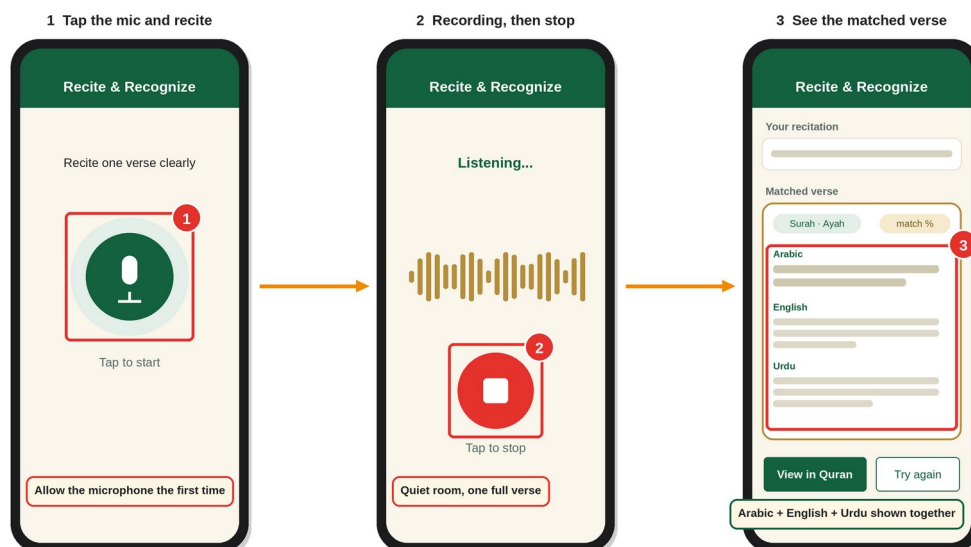


Illustration of the screen layout (example content) – your phone shows the real verse text.

Figure 14: Recite & Recognize: record, stop, and read the matched verse in three languages

10.7 Bookmarks and history

Both use Hive, so they survive app restarts and work offline.

10.8 Settings

Theme (Classic light/dark, Blossom, Dracula, Nord, Gruvbox, Catppuccin Mocha, Solarized Dark, Monokai, Tokyo Night, Rosé Pine, or follow the system), interface language, Arabic font size, show Arabic, show translation, translation language, reciter, playback speed, **Auto Play**, response language, voice responses.

10.9 Accounts

Login and Signup are **local, on-device accounts** (the backend has no authentication endpoints). They control app flow but are not real security; see section 15.

11. Data stored on the phone

Hive box	Contents
<code>quran_cache</code>	Bundled offline Quran plus anything fetched from the backend
<code>bookmarks_box</code>	Saved verses
<code>history_box</code>	Chat history
<code>app_flags_box</code>	<code>isFirstLaunch</code> , <code>isLoggedIn</code>
<code>settings_box</code>	All user settings and last-read position

Recitation MP3s are stored as files in the app's support folder under `recitations/`. Uninstalling the app deletes all of this.

12. Running, building and sharing the APK

12.1 Run on a phone (debug)

After the one-time setup in sections 6.4 and 6.5 and setting the backend URL (section 9):

```
cd flutter_app
flutter pub get
flutter run
```

While running: **r** = hot reload, **R** = hot restart, **q** = quit.

12.2 Build the release APK

```
flutter build apk --release
```

Output: **build\app\outputs\flutter-apk\app-release.apk** (roughly 55 MB). The first release build is slow because Gradle downloads and compiles everything; later builds are faster.

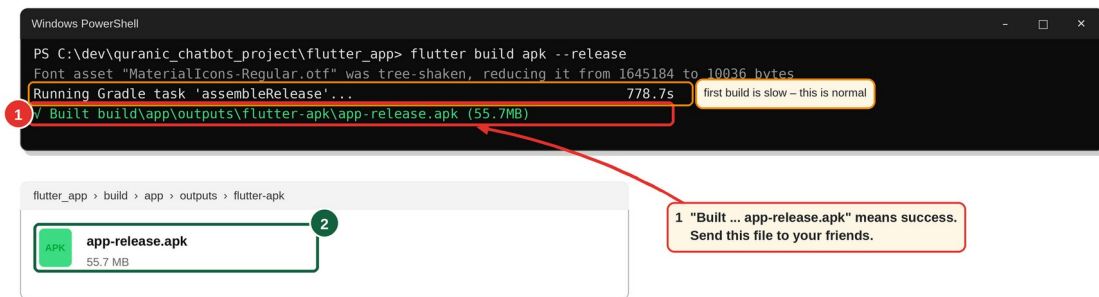


Figure 15: A successful release build and where to find the APK

Smaller files, one per phone architecture:

```
flutter build apk --release --split-per-abi
```

Most modern phones need **app-arm64-v8a-release.apk**.

12.3 Sending the APK to someone

- Send the **.apk** by WhatsApp, Google Drive or a USB cable.
- The receiver must allow **Install unknown apps** for the app they open it with.
- Whenever the app is updated, send the new APK; older installs do not update themselves.
- The app works only while the Vercel backend is running and the API keys are valid.

13. Troubleshooting

Problem	Cause and fix
WARNING: A restricted method in java.lang.System has been called	Gradle 9 notice. Harmless. Ignore.
SDK XML versions up to 3 ... version 4	Android Studio and the command-line tools were released at different times. Harmless. Update both in SDK Manager if you want it gone.

Problem	Cause and fix
Unable to delete directory ...\\build\\app\\intermediates\\...	Windows has the build folder locked. Run .\\android\\gradlew.bat --stop, taskkill /F /IM java.exe, flutter clean , then flutter run . Close any Explorer window open inside build . Add the project folder to Windows Security exclusions, or keep the project outside Downloads (for example C:\\dev\\). Restart the PC if needed.
Source value 8 is obsolete (Java warnings)	Dependency warnings. Harmless.
First Gradle build is very slow	Normal (downloads Gradle and dependencies).
The argument type 'Widget?' can't be assigned to 'Widget'	A builder's nullable child is passed where a non-null widget is required. Use child! if the builder always supplies it.
App shows no chat answer / a fallback sentence	Both AI providers failed. Check the Vercel Logs for [GROQ ERROR] / [GEMINI ERROR] : wrong or missing key, retired model name, or rate limit.
Vercel deployment crashes at start	GROQ_API_KEY or GEMINI_API_KEY is missing. Add it, then Redeploy (section 8.4).
Recite & Recognize is slow the first time	Backend cold start plus loading the whole Quran text. Try again; later requests are faster.
Recite & Recognize returns "no match"	Recite a full verse, reduce noise, speak clearly.
Recognized verse shows no English/Urdu	The deployed api/index.py must return english and urdu in /recognize . Redeploy the latest backend.
Unable to find asset / "asset ... not found"	The assets: lines in pubspec.yaml were uncommented before assets/quran_offline.json existed. Build the file (section 6.6) or put the # back.
minSdkVersion 21 cannot be smaller than version 23	The record package needs Android 6+. Set minSdk = 23 in android/app/build.gradle (or .kts).
App opens but every list shows an error	kBackendBaseUrl in lib/services.dart still has the placeholder, or the backend is down. Set it (section 9) and test /surahs in a browser.
Vercel build error about missing packages	requirements.txt must sit at the top level of the repository next to vercel.json .
Verse won't play	Check internet on the first play of that verse (it downloads then). Check the [audio] lines in the flutter run console.
Auto Play does nothing	It is off by default. Turn it on in Settings.
Microphone doesn't record	Grant the permission in phone Settings → Apps → Quranic Chatbot → Permissions.
Flutter can't find the phone	Reconnect USB, accept the debugging prompt, run flutter devices , try another cable.

14. Security and religious-content notes

Security

- API keys only in Vercel environment variables. Never in code, GitHub or chat.
- The backend allows requests from any origin (`allow_origins=["*"]`). That is fine for a student project; for production, restrict it and add rate limiting so nobody can use up your free-tier quota.
- Local login is not a real authentication system.

Religious content

- All Quran text, translations and audio come from established public sources (alquran.cloud, everyayah.com), not from AI.
- Verse recognition uses text similarity against the real Quran, so it cannot invent verse numbers.
- **AI chat answers are generated by language models and can be wrong.** Citations should be checked against the Quran, and important religious rulings should be confirmed with a qualified scholar. Keep a short disclaimer like this in the About screen.
- Mock or sample content must stay clearly separate from verified source content.

15. Known limitations and future work

- **Text-to-speech is simulated.** Voice replies are not actually spoken. A real TTS package (or a backend TTS endpoint) would be needed.
- **Reciters:** the backend builds every audio URL for one reciter (Alafasy, 128 kbps). To offer the other reciters listed in Settings, the backend must return audio URLs for them.
- **Local accounts** are not secure and are not shared between devices. Real accounts would need backend authentication (or Firebase Auth).
- **Cold starts** on Vercel's free plan make the first request after idle time slow.
- **Serverless time limits** on the free plan can cut off very slow requests.
- **Free-tier quotas** on Groq and Gemini can run out under heavy use.
- **No iOS build** has been prepared (Android first).
- Ideas: more reciters, whole-Surah offline audio download, word-by-word view, tafsir, hadith (with verified sources only), a proper disclaimer screen, automated tests.

16. Quick command reference

First-time project setup (inside flutter_app)

```
flutter create . --platforms=android --project-name quranic_chatbot
python tools/add_android_permissions.py
flutter pub get
```

Flutter

```
flutter doctor -v          # check setup
flutter devices            # list phones/emulators
```

```
flutter pub get           # install packages
flutter run               # run on phone (debug)
flutter clean             # wipe build folder
flutter build apk --release # release APK
flutter build apk --release --split-per-abi
```

Windows build-lock fix

```
.\android\gradlew.bat --stop
taskkill /F /IM java.exe
flutter clean
```

Backend (local)

```
cd backend
pip install -r requirements.txt
pip install uvicorn
uvicorn api.index:app --reload
```

Git (push backend changes to trigger a Vercel deploy; run inside the backend repository)

```
git add .
git commit -m "Update backend"
git push
```